# Guide technique

Architecture, modèles de données et extension

Module **fuel_station** — Tryton 7.0 — version 7.0.3

# 1. Vue d'ensemble

Le module **fuel_station** gère l'exploitation d'une ou plusieurs stations-service : cuves, pompes et pistolets, prix des carburants, quarts de travail avec relevé des index de compteurs, livraisons de carburant, jaugeages et analyse des ventes. Il ne recrée pas de gestion de stock : **chaque cuve est un emplacement de stock natif** et chaque vente, livraison ou ajustement est un mouvement `stock.move` terminé.

## 1.1 Dépendances

| Module | Usage |
|---|---|
| company | Société, employés (pompistes), règles multi-sociétés |
| currency | Devise de la société, arrondis des montants |
| party | Fournisseurs de carburant, contact de la station |
| product | Carburants (article avec case « Carburant »), unités (gallon), prix de revient |
| stock | Entrepôts, emplacements (cuves), mouvements, valorisation |
| ir, res | Séquences, accès, groupes, traductions, icônes |

Le module principal n'exige ni *sale* ni *account*. La facturation des ventes à crédit est fournie par le module compagnon **fuel_station_invoice** (dépend de account, account_invoice, account_product et fuel_station) : il est facultatif.

## 1.2 Organisation des fichiers

| Fichier | Contenu |
|---|---|
| station.py | Station, Cuve, Pompe, Pistolet |
| price.py | Prix des carburants et recherche du prix applicable |
| shift.py | Quart de travail (workflow), lignes par pistolet, paiements, extension de stock.move (origines) |
| delivery.py | Livraison de carburant (réception en cuve) |
| dip.py | Jaugeage de cuve, écart et ajustement de stock |
| report.py | Analyse des ventes (table_query) et rapport de quart (.fodt) |
| product.py | Case « Carburant », famille, densité sur product.template ; unité gallon par défaut et contrôle de la catégorie Volume |
| party.py | Case « Personne physique » et champ « Prénom » sur party.party ; nom affiché « Prénom Nom » |
| cash.py | Coupures (billets/pièces), comptage de caisse par quart, assistant de création des coupures standard |
| meter.py | Relevés de compteur (API record(), assistant d'import CSV) |
| shop.py | Ventes boutique / GPL du quart |
| authorization.py | Autorisations de distribution : demande, validation, interface middleware, journal d'audit, expiration (cron), blocage |
| guide.py | Assistant « Guide d'utilisation » du menu (texte traduisible) |
| tools.py | Facteur de correction de volume à 15 °C (ASTM D1250) |
| *.xml, view/ | Menus, actions, vues, droits, règles, séquences, messages, icônes |
| locale/fr.po | Traduction française complète |
| tests/ | 56 tests (ModuleTestCase) ; chaque module compagnon a son test |

# 2. Modèle de données

| Modèle | Rôle et champs principaux |
|---|---|
| fuel.station | société, code (unique par société), nom, tiers, entrepôt (type warehouse), tolérance de caisse, tolérance de jaugeage (%), comptage de caisse obligatoire, comptabilisation à 15 °C |
| fuel.tank | station, code (unique par station), carburant (product.product avec template.fuel), capacité, niveau d'alerte, emplacement de stock (créé automatiquement), quantité, remplissage, bas (champs calculés) |
| fuel.pump | station, code (unique par station), nom, pistolets |
| fuel.nozzle | pompe, numéro (unique par pompe), cuve (de la même station), index du compteur (totalisateur) |
| fuel.price | société, station (facultatif), carburant, date d'effet, prix (arrondi selon price_decimal) |
| fuel.shift | numéro (séquence), station, pompiste (company.employee), date, début/fin, état, lignes, paiements, totaux |
| fuel.shift.line | quart, pistolet, index d'ouverture/fermeture, quantité d'essai, quantité vendue, prix, montant, mouvement |
| fuel.shift.payment | quart, mode (espèces, carte, argent mobile, bon, crédit), montant, référence, client (obligatoire à crédit) |
| fuel.shift.sale | vente boutique/GPL : quart, article (non carburant), quantité, prix, montant, mouvement de stock |
| fuel.denomination | devise, type (billet/pièce), valeur ; unique par (devise, type, valeur) |
| fuel.shift.cash.count | quart, coupure, quantité comptée, quantité suspecte (faux/abîmés), montants |
| fuel.authorization | numéro (AU/), station, pompe, pistolet, limite (quantité ou montant), prix, client, véhicule, état, demandeur/valideur et heures, expiration, référence contrôleur, quantité et montant distribués, quart, mouvement |
| fuel.authorization.log | journal d'audit non modifiable : événement, utilisateur, état, note, heure |
| fuel.meter.reading | pistolet, date et heure, index, source (manuel, import, automate) |
| fuel.delivery(.line) | numéro, station, fournisseur, bon de livraison, chauffeur, lignes (cuve, quantité reçue, température, densité, quantité à 15 °C, coût unitaire, mouvement) |
| fuel.dip | numéro, cuve, date, quantité mesurée, quantité comptable, écart, ajustement, mouvement |
| fuel.report.sales | vue SQL en lecture seule : date, station, carburant, quantité, montant des quarts fermés |

## 2.1 Contraintes

- Unicité SQL : code de station par société, code de cuve par station, code de pompe par station, numéro de pistolet par pompe, prix par (société, station, carburant, date), pistolet par quart.
- Contrôles SQL : niveau d'alerte ≤ capacité ; index de fermeture ≥ index d'ouverture.
- Domaines : la cuve d'un pistolet, d'une livraison ou d'un jaugeage appartient à la station ; validations Python en secours.
- Une seule station ouverte par quart à la fois (un seul quart « Ouvert » par station).

# 3. Intégration au stock natif

| Opération | Mouvement | Origine |
|---|---|---|
| Vente (fermeture de quart) | cuve → emplacement client, quantité = vendue, prix = prix du quart | fuel.shift.line |

| Opération | Mouvement | Origine |
|---|---|---|
| Livraison reçue | emplacement fournisseur du tiers → cuve, prix = coût unitaire | fuel.delivery.line |
| Vente boutique/GPL (fermeture de quart) | emplacement de stockage de l'entrepôt → emplacement client | fuel.shift.sale |
| Jaugeage avec ajustement | perte : cuve → « Pertes et profits » ; gain : « Pertes et profits » → cuve, valorisé au prix de revient | fuel.dip |

La quantité d'une cuve est lue avec `Product.products_by_location([location])` : c'est la quantité réelle en stock à la date du jour. Les mouvements sont créés dans un contexte `without_check_access()` pour qu'un pompiste n'ait pas besoin des droits du groupe Stock.

# 4. Workflows

## 4.1 Quart de travail

```
Brouillon --Ouvrir--> Ouvert --Fermer--> Fermé
    |                   |
    +------Annuler-------+--> Annulé --Brouillon--> Brouillon
```

- **Ouvrir** : vérifie qu'aucun autre quart n'est ouvert pour la station, crée une ligne par pistolet (index d'ouverture = index courant du pistolet, prix = prix applicable à la date du quart), enregistre l'heure de début.
- **Fermer** : exige l'index de fermeture de chaque ligne ; quantité vendue = fermeture – ouverture – essai ; montant = quantité × prix, arrondi à la devise ; refuse une quantité négative ; demande une confirmation (`UserWarning`) si |payé – ventes| dépasse la tolérance de caisse ; crée et exécute les mouvements ; reporte l'index de fermeture dans le pistolet ; enregistre l'heure de fin.
- Un quart fermé ne peut être ni modifié (paiements), ni supprimé.

## 4.2 Livraison et jaugeage

- Livraison : Brouillon → Reçue (mouvements) ou Annulée. Avertissement si la quantité dépasse la place libre de la cuve.
- Jaugeage : Brouillon → Validé. Fige quantité comptable et écart ; avertissement si l'écart dépasse la tolérance de la station ; ajustement optionnel du stock.

## 4.3 Contrôle de caisse (billets et pièces)

- Les **coupures** (fuel.denomination) sont définies par devise ; l'assistant « Créer les coupures standard » charge HTG, USD, EUR, CAD et DOP.
- Pendant un quart ouvert, le bouton **Charger les coupures** crée une ligne de comptage par coupure active ; le pompiste saisit la quantité **comptée** et, séparément, la quantité **suspecte** (faux billets, billets ou pièces abîmés).
- Espèces comptées = Σ (comptées – suspectes) × valeur ; **écart de caisse** = espèces comptées – fonds de caisse – paiements en espèces.
- À la fermeture : comptage obligatoire si la station l'exige (erreur) ; confirmation si des suspects existent ; confirmation si |écart| dépasse la tolérance de caisse.

## 4.4 Relevés de compteur

- `MeterReading.record(station, pompe, numéro, index, source='device')` enregistre un relevé : appelable par RPC depuis un automate/concentrateur de pompes.

- Assistant d'import CSV (station, pompe, pistolet, index) ; le bouton **Récupérer les relevés** du quart reporte le dernier relevé postérieur à l'ouverture dans l'index de fermeture (erreur s'il est inférieur à l'ouverture).

## 4.5 Correction de température et densité

Facteur de correction : VCF = exp(–α·ΔT·(1 + 0,8·α·ΔT)), avec α = K0/ρ² + K1/ρ (ρ en kg/m³ à 15 °C, ΔT = T – 15 °C), coefficients ASTM D1250 pour l'essence, le diesel et le kérosène/jet. Les autres familles ne sont pas corrigées (facteur 1). Si la station coche « Comptabiliser à 15 °C », les livraisons et jaugeages avec température saisie sont enregistrés (stock) au volume corrigé ; sinon le facteur n'est qu'informatif.

## 4.6 Boutique, GPL et facturation à crédit

- Onglet « Boutique et GPL » du quart : articles non carburant (lubrifiants, bouteilles de GPL...) ; mouvement de stock de l'entrepôt vers le client à la fermeture ; montant inclus dans le total des ventes.
- Module **fuel_station_invoice** : à la fermeture, les paiements « Crédit client » créent une **facture client brouillon** par client (une ligne par quart, compte de produits pris sur la catégorie comptable du carburant). Il faut un compte de produits sur le carburant et une adresse sur le client, sinon une erreur explicite est levée.

## 4.8 Autorisations de distribution

**Principe de sécurité.** Tryton demande, valide et enregistre ; le **contrôleur certifié de la station décide physiquement** si une pompe peut distribuer. Tryton n'est **jamais** branché sur les circuits de commande des pompes : un middleware, dont l'interface avec l'équipement doit être validée par le fabricant et réalisée par un installateur habilité, échange uniquement des autorisations et des résultats avec Tryton (JSON-RPC).

| État | Transitions possibles |
|---|---|
| En attente | Valider → Validée ; Rejeter → Rejetée ; Annuler → Annulée |
| Validée | Middleware : transmise → Transmise ; expiration → Expirée ; Annuler → Annulée |
| Transmise | Contrôleur : confirme → Confirmée ; refuse → Refusée ; expiration → Expirée ; Annuler (responsable) → Annulation demandée |
| Confirmée | Résultat conforme → Terminée ; résultat incohérent → Écart ; Annuler (responsable) → Annulation demandée |
| Annulation demandée | Acquittement du contrôleur → Annulée |
| Expirée / Annulée | Confirmation ou résultat tardif du contrôleur → Écart |
| Écart | Traiter (responsable) → Traitée |

| Règle | Mise en œuvre |
|---|---|
| Usage unique | Un seul résultat accepté ; un second est refusé et journalisé ; une transaction ne peut être liée qu'à une autorisation |
| Expiration courte | Validité de la station (5 min par défaut) à partir de la validation ; tâche planifiée chaque minute et contrôle à la transmission ; une confirmation tardive donne un écart |
| Plafond | Quantité maximale (gallons) ou montant maximal ; l'autre limite est calculée avec le prix du jour (arrondi vers le bas pour la quantité) ; tout dépassement donne un écart, rien n'est comptabilisé |
| Pompe et carburant imposés | Le pistolet choisi fixe la pompe, le carburant et la cuve ; une seule autorisation active par pompe |
| Séparation des tâches | Le valideur est différent du demandeur et appartient au groupe « Responsable des autorisations carburant » |

| Règle | Mise en œuvre |
|---|---|
| Annulation contrôlée | Demandeur ou responsable avant transmission ; après transmission seul un responsable demande, le contrôleur doit acquitter ; pas de suppression |
| Journal d'audit | Modèle fuel.authorization.log (non modifiable, non supprimable) : demande, validation, transmission, confirmation, résultat, écarts, expiration, annulation, avec utilisateur et heure |
| Blocage | Aucune demande, validation ou transmission si le dernier signal du middleware est plus ancien que le délai de la station, ou si le blocage manuel est coché ; le middleware envoie un signal à chaque cycle |

**Interface du middleware** (méthodes RPC, groupe « Middleware carburant » obligatoire) :

| Méthode | Rôle |
|---|---|
| fuel.station.heartbeat(station, société) | Signal de vie |
| fuel.authorization.middleware_pending(station, société) | Autorisations validées à transmettre (liste vide si communication perdue) ; compte aussi comme signal de vie |
| …middleware_transmitted(numéro) | Autorisation remise au contrôleur (refusée si expirée) |
| …middleware_confirmed(numéro, réf) | Le contrôleur a confirmé (écart si l'autorisation n'est plus valable) |
| …middleware_refused(numéro, motif) | Le contrôleur a refusé |
| …middleware_result(numéro, quantité, montant, transaction) | Données de distribution ; rapprochées de l'autorisation |
| …middleware_cancel_ack(numéro) | Annulation acquittée par le contrôleur |

Un exemple de middleware (`middleware/fuel_middleware.py`) fournit le client JSON-RPC, la boucle de cycle et une classe `ControllerAdapter` à implémenter avec l'interface documentée par le fabricant. Seul un contrôleur **simulé** est fourni pour les essais.

**Comptabilisation.** À la fin (Terminée) : mouvement de stock cuve → client à la quantité réelle, rattachement au quart ouvert ; à la fermeture du quart, ces gallons ne sont pas déplacés une seconde fois (index du compteur – gallons autorisés) et les ventes autorisées comptent dans le rapprochement (Différence = payé + ventes autorisées – ventes). Les distributions sans autorisation ou en écart restent captées par les index du compteur.

### 4.9 Ventes et factures : module fuel_station_sale

Dépend de *sale*. À la fin d'une autorisation, il crée une **vente** (client de l'autorisation, sinon « Client par défaut » de la station) avec une ligne au prix autorisé et à la quantité distribuée, méthode d'expédition « manuelle » (le stock est déjà mouvementé), méthode de facturation « à la commande » : la vente est confirmée et traitée, la **facture client brouillon** est créée. Sans client, l'autorisation est terminée mais aucune vente n'est créée (message dans le journal). Les carburants cochés « Carburant » deviennent vendables, unité de vente = gallon. **Le module Sale Point (ventes au point de vente) n'est pas intégré** : les ventes en caisse POS restent à brancher séparément.

### 4.7 Unité de mesure

Le gallon (gallon US, 3,785411784 L) est l'unité par défaut des carburants : la case « Carburant » force l'unité par défaut au gallon, et le module refuse une unité hors de la catégorie Volume. Tous les libellés (« Gallons vendus ») et champs de quantité (capacité, niveaux, index, livraisons, jaugeages) sont exprimés dans l'unité de l'article.

## 5. Droits, groupes et règles

| Groupe | Droits |
|---|---|
| Station-service | Lecture de la configuration ; création/modification/suppression des quarts, paiements, livraisons, jaugeages ; lecture de l'analyse des ventes |
| Administration station-service | Tous les droits sur tous les modèles du module (hérite du groupe précédent) |

Chaque modèle possède une règle globale « Utilisateur dans les sociétés » (`ir.rule`) : un utilisateur ne voit que les données des sociétés auxquelles il est rattaché. Les séquences (quart, livraison, jaugeage) sont lisibles par les deux groupes.

# 6. Prix

`Price.get_price(station, produit, date)` renvoie le prix de la station le plus récent dont la date d'effet est ≤ date ; à défaut, le prix de la société (station vide) ; à défaut, une erreur explicite. Le prix est arrondi par `round_price` du module product : il respecte donc le paramètre *price_decimal* de trytond.conf.

# 7. Rapports et vues

- **Rapport de quart** (`fuel.shift.report`, shift.fodt) : action « Imprimer » du formulaire de quart ; lignes par pistolet, paiements, totaux, différence.
- **Analyse des ventes** : modèle SQL `fuel.report.sales`, liste avec totaux et graphique.
- **Niveaux des cuves** : liste avec barre de remplissage, lignes en rouge sous le niveau d'alerte ; menu « Cuves basses ».
- Convention de l'interface : aucun champ datetime dans les listes (commentés dans les vues « list »).

## 7.1 Ergonomie

- Formulaires structurés en groupes et onglets (Relevés des pistolets, Boutique et GPL, Paiements, Contrôle de caisse, Notes) avec totaux regroupés.
- Listes colorées selon l'état (quart ouvert en alerte, fermé en succès, annulé estompé), totaux en pied de liste, barre de remplissage des cuves.
- Valeurs par défaut : société, date, employé de l'utilisateur, station si elle est unique, unité gallon, fonds de caisse à 0.
- Boutons d'action dans le formulaire du quart (Récupérer les relevés, Charger les coupures), icônes dédiées (billets, boutique).
- Champ « Prénom » affiché seulement pour les personnes physiques ; recherche des tiers par prénom.

# 8. Extension

- Facturation du carburant au comptant ou par vente : étendre `fuel.shift` comme le fait fuel_station_invoice.
- Comptabilité : les mouvements portent déjà prix et origine ; le module *account_stock_continental* peut les valoriser.
- Autres modes de paiement : ajouter une valeur à `PAYMENTS` par extension de la sélection de `fuel.shift.payment.method`.

- Origines de mouvement : `stock.move._get_origin` est étendu pour fuel.shift.line, fuel.delivery.line et fuel.dip.

# 9. Tests

46 tests (dont ceux de ModuleTestCase : vues, accès, règles, dépendances, contextes multi-valeurs) :

```
export DB_NAME=:memory:
python -m unittest trytond.modules.fuel_station.tests.test_module.FuelStationTestCase
```

Autorisations : flux complet, limites en montant, séparation des tâches, communication perdue, désactivation, écart et usage unique, expiration, pompe occupée, annulation contrôlée, stock avec quart, guide. Un essai réel a aussi été fait contre un serveur trytond (JSON-RPC) avec le middleware d'exemple et un contrôleur simulé : autorisation Terminée de bout en bout. Les tests couvrent aussi : prénom du tiers, unité gallon, facteur de correction, livraison à 15 °C, coupures standard, contrôle de caisse (écart, suspects, obligation), crédit sans client, ventes boutique, relevés automatiques. Le module fuel_station_invoice a son propre test (facture brouillon par client). Couvrent : création d'emplacement de cuve, quantités et alerte, prix station/société, cycle complet d'un quart (ouverture, essais, paiements, fermeture, mouvements, index), un seul quart ouvert, indexes invalides, avertissement de différence de caisse, livraison et dépassement de capacité, cuve d'une autre station, jaugeage (écart, tolérance, ajustement perte/gain), analyse des ventes, rendu du rapport.

# 10. Limites connues

Autorisations : aucun contrôle de pompe n'est fourni ni testé avec un vrai contrôleur ; l'adaptateur du fabricant reste à écrire et à faire valider.

Limites levées dans une version précédente, avec leur périmètre exact :

- **Relevés de compteur** : API et import CSV, pas de protocole direct d'automate (PTS-2, Wayne DART, etc.) : un connecteur externe doit appeler MeterReading.record().
- **Facturation** : factures clients *brouillon* pour les ventes à crédit uniquement (module compagnon) ; pas de taxes ajoutées ni de validation automatique ; les paiements comptant/carte ne génèrent aucune écriture comptable.
- **Température/densité** : facteur ASTM D1250 simplifié pour essence, diesel, kérosène ; pas de tables complètes (huiles, GPL) ; température saisie à la main.
- **Boutique/GPL** : vente d'articles-stock de l'entrepôt ; pas d'alerte de stock insuffisant, pas de consigne ni d'échange de bouteilles, pas de caisse enregistreuse.

Limites restantes :

- Gallon US par défaut : si vous utilisez le gallon impérial, créez l'unité et choisissez-la sur les articles.
- Tests exécutés sur SQLite ; PostgreSQL recommandé en production mais non exécuté ici.
- Les écrans n'ont pas été vérifiés visuellement dans le client Tryton (structure et schémas validés par les tests).

# Station-service : architecture des liaisons

Mise à jour v3 — ajout au guide d'origine. Auteur : Pierre Michel Augustin — AGPLv3.

| Module | Réutilise |
|---|---|
| fuel_station_cash_control | CountMixin et coupures de sale_point_cash_control |
| fuel_station_sale_point | sale_point et sale_point_quick (boutique) |
| fuel_station | party_person_ht (prénom, NIF, NINU) |

Correction apportée à sale_point_cash_control : la clôture avec un écart confirmé recalculait mal le solde de la session (cache) ; elle réinstancie maintenant les sessions après l'écriture de l'écart. 32 tests.